



JAL1501 MACHINE LEARNING

UNIT III ENSEMBLE TECHNIQUES AND UNSUPERVISED LEARNING

UNIT III ENSEMBLE TECHNIQUES AND UNSUPERVISED LEARNING

SYLLABUS

Combining multiple learners: Model combination schemes, Voting, Ensemble Learning bagging, boosting, stacking.

Unsupervised learning: K-means, Instance Based Learning: KNN, Gaussian mixture models and Expectation maximization.



UNSUPERVISED LEARNING

- K-means
- Instance Based Learning: KNN
- Gaussian mixture models
- Expectation maximization

Unsupervised Learning

- A type of machine learning where the model is trained on **data without labeled outcomes**.
- The system tries to learn the underlying structure of the data without explicit supervision.
- The goal → to find hidden patterns or intrinsic structures in the input data.

Key Concepts

- **No labels or targets** → algorithm is not provided with any ground truth during training
- **Clustering** → One of the most common techniques in unsupervised learning → involves grouping data points based on their similarities
- **Dimensionality Reduction** → used to reduce the number of features while retaining the important information
- **Association** → used to discover interesting relationships between variables in large datasets

Unsupervised Learning

Challenges

- No labels → hence evaluating the performance of unsupervised learning models can be tricky.
- Results might be less interpretable compared to supervised learning models.

Applications - Plays a crucial role in discovering insights from data when there is no prior knowledge of the labels or structure.

- Market Segmentation (based on purchasing behaviors)
- Anomaly Detection (unusual data points)
- Recommendation Systems (Suggesting based on the similarities)
- Dimensionality Reduction for Visualization (Reducing large datasets into 2D or 3D)

Real Life examples of Unsupervised Learning

•Audience segmentation.

•Customer persona investigation.

•Anomaly detection (for example, to detect bot activity)

•Pattern recognition (grouping images, transcribing audio)

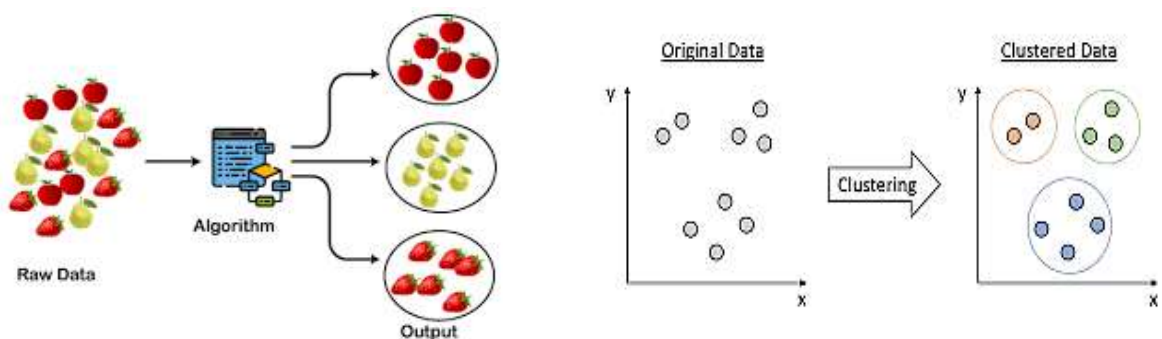
•Inventory management (by conversion activity or by availability)

•Optical Character recognition (including handwriting recognition)

Clustering

- Cluster → a group of objects that belongs to the same class.
- Clustering → The task of grouping a set of objects in such a way that objects in the same group (called a cluster) are more similar (in some sense) to each other than to those in other groups (clusters).
- Clustering → process of partitioning a set of data in a set of meaningful subclasses. Every data in the sub class shares a common feature / behavior - helps a user understand the natural grouping or structure in a data set.
- A cluster → a collection of objects which are “similar” between them and are dissimilar” to the objects belonging to other clusters.
- Powerful data-mining tool for any organization that needs to identify discrete groups.
- Types of clustering methods → partitioning methods, hierarchical clustering, fuzzy clustering, density based clustering and model based clustering.

Clustering



Clustering

- **Different clustering algorithms**
- K means Clustering → divides dataset into a specified number (k) of clusters
- Hierarchical Clustering → creates a hierarchy of clusters
- DBSCAN (Density-Based Spatial Clustering of Applications with Noise) → identifies clusters of points in a dataset based on their density.
- Expectation-Maximization (EM) Algorithm → identifies patterns and structures in data by fitting a probabilistic model to the data.
- Spectral Clustering → based on the eigenvectors of the data's similarity matrix.

Clustering

How clustering algorithms work?

- Find natural groups components or data based on some similarity.
- The clustering algorithm finds the centroid of a group of data sets.
- Cluster centroid → a point whose parameter values are the means of the parameter values of all the points in the cluster. Each cluster has a well defined centroid.
- Most algorithms evaluate the distance between a point and the cluster centroids.
- Distance → a common metric to see the similarity among the components of population → commonly used one is Euclidean metric
- Distance between two points $p = (p_1, p_2, \dots, p_k)$ and $q = (q_1, q_2, \dots, q_k)$ is given by, $D = \sum_{i=1}^k (p_i - q_i)^2$
- Output from a clustering algorithm → a statistical description of the cluster centroids with the number of components in each cluster.

Unsupervised Learning

- **K-means algorithm**

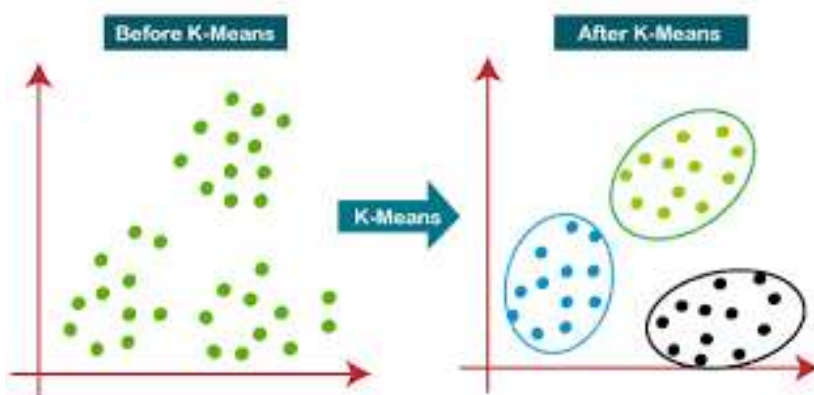
K-means Algorithm

- K-Means Clustering is an unsupervised learning algorithm that is used to solve the clustering problems in machine learning or data science.
- Groups the unlabeled dataset into K different clusters.
- Here K defines the number of pre-defined clusters that need to be created in the process; Eg. if $K=2$, there will be two clusters, and for $K=3$, there will be three clusters, and so on.
- **An iterative algorithm that divides the unlabeled dataset into k different clusters in such a way that each dataset belongs only one group that has similar properties.**
- Allows us to cluster the data into different groups and a convenient way to discover the categories of groups in the unlabeled dataset on its own without the need for any training.

K-means Algorithm

- A centroid-based algorithm, where each cluster is associated with a centroid.
- The main aim of this algorithm → **to minimize the sum of distances between the data point and their corresponding clusters.**
- The algorithm takes the unlabeled dataset as input, divides the dataset into k-number of clusters, and repeats the process until it does not find the best clusters.
- The value of k should be predetermined in this algorithm.
- The k-means clustering algorithm mainly performs two tasks:
 - ❑ Determines the best value for K center points or centroids by an iterative process.
 - ❑ Assigns each data point to its closest k-center. Those data points which are near to the particular k-center, create a cluster.
- Hence each cluster has data points with some commonalities, and it is away from other clusters.

K-means Algorithm



K-means Algorithm

Steps of the K-Means Algorithm:

1. **Choose the number of clusters (K)** → decide how many clusters is needed for the data to be divided into, which is the parameter K.
2. **Initialize Centroids** → Randomly select K data points from the dataset. These data points will act as the initial centroids of the clusters. Each centroid represents the center of a cluster.
3. **Assign Data Points to the Nearest Cluster** → For each data point, calculate its distance to each of the K centroids. Assign the data point to the cluster whose centroid is the closest to it (typically using Euclidean distance).

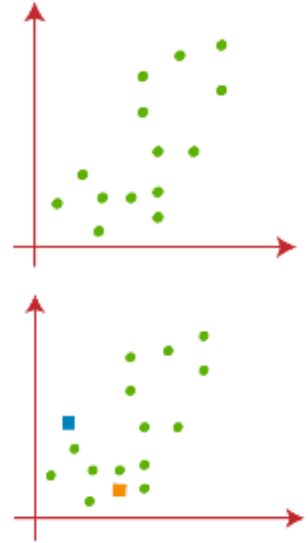
K-means Algorithm

Steps of the K-Means Algorithm (contd.)

4. **Update Centroids** → After all data points have been assigned to clusters, recompute the centroid of each cluster. The centroid is the mean of all the points in the cluster (average position of the points).
5. **Repeat** → Repeat the process of assigning points to clusters and updating the centroids until the centroids no longer change (or the changes are very minimal). This means that the clusters have converged.
6. **Stop the Algorithm** → The algorithm stops when the centroids stabilize, meaning the clusters are no longer changing significantly or a predefined number of iterations is reached.

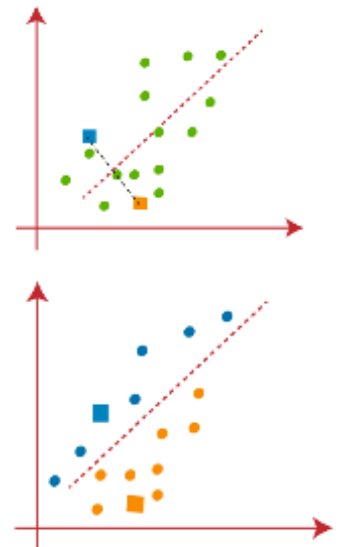
K-means Algorithm

- Suppose we have two variables M1 and M2. The x-y axis scatter plot of these two variables is given:
- Let's take number k of clusters, i.e., $K=2$, to identify the dataset and to put them into different clusters → try to group these datasets into two different clusters.
- Choose some random k points or centroid to form the cluster. These points can be either the points from the dataset or any other point.
- Here two points as (k points) are selected, which are not the part of our dataset.



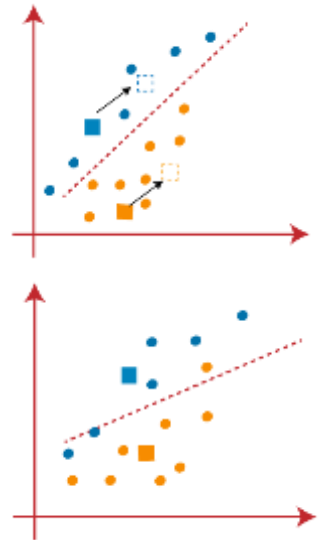
K-means Algorithm

- Now assign each data point of the scatter plot to its closest K-point or centroid.
- Compute the distance by applying some mathematics formula like Euclidean metric to calculate the distance between two points. So, we will draw a median between both the centroids. Consider the image:
- It is clear that points left side of the line is near to the K1 or blue centroid, and points to the right of the line are close to the yellow centroid. They can be differentiated as blue and yellow for clear visualization.



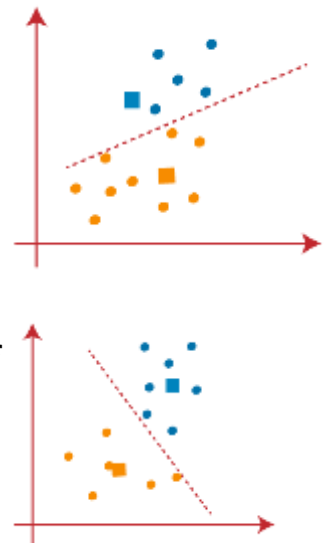
K-means Algorithm

- As we need to find the closest cluster, the process will be repeated by choosing a new centroid.
- To choose the new centroids, compute the center of gravity of these centroids (mean), and will find new centroids:
- Now reassign each datapoint to the new centroid. And repeat the same process of finding a median line. The median will be like the one given in image:
- Now one yellow point is on the left side of the line, and two blue points are right to the line.



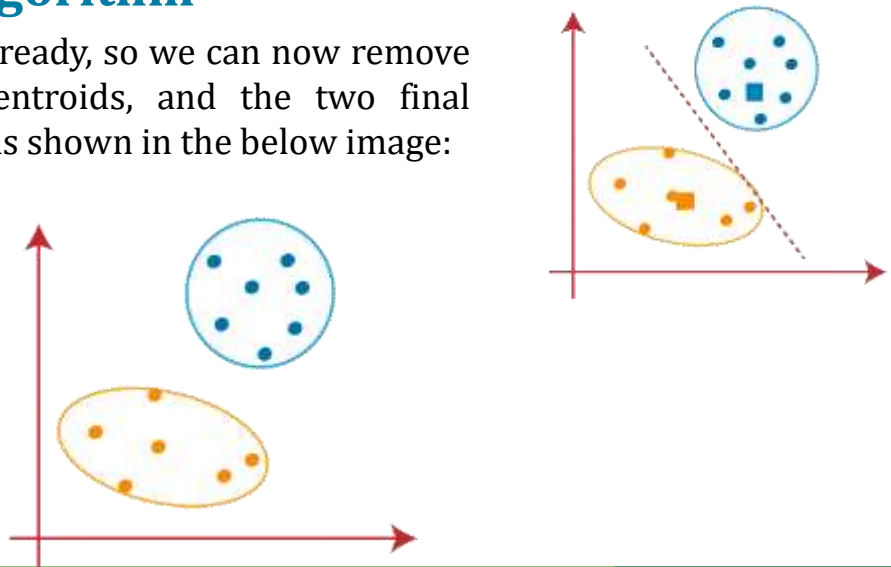
K-means Algorithm

- So, these three points will be assigned to new centroids.
- As reassignment has taken place, repeat finding new centroids or K-points by finding the center of gravity of centroids, so the new centroids will be as shown in the image:
- There are no dissimilar data points on either side of the line, which means our model is formed.



K-means Algorithm

- As our model is ready, so we can now remove the assumed centroids, and the two final clusters will be as shown in the below image:



K-means Algorithm

Mathematical Representation

- Centroid Calculation: For each cluster C_i , the centroid μ_i is the mean of all points x_j in that cluster:

- Here $|C_i|$ is the number of data points in cluster C_i .

$$\mu_i = \frac{1}{|C_i|} \sum_{x_j \in C_i} x_j$$

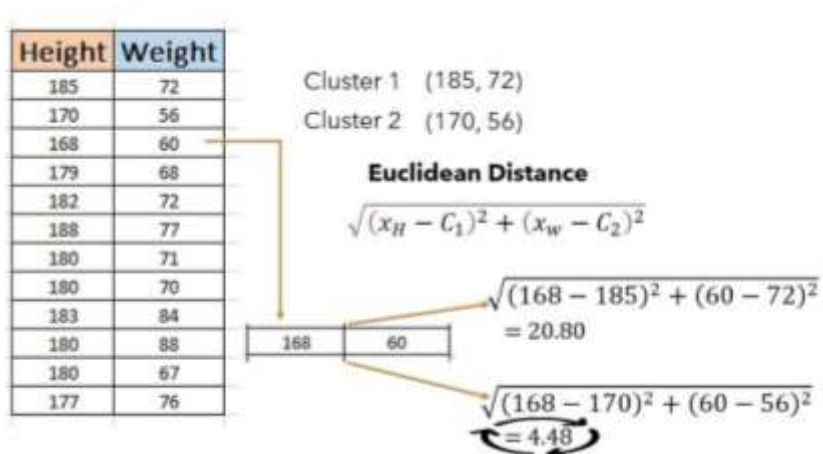
- Objective: The goal of K-Means is to minimize the following objective function (known as **inertia**)

$$\text{Inertia} = \sum_{i=1}^K \sum_{x_j \in C_i} \|x_j - \mu_i\|^2$$

- This equation minimizes the sum of the squared distances between each point and the centroid of its assigned cluster

K-means Algorithm

- Example problem:



K-means Algorithm

Example problem:

- Apply K(=2)-Means algorithm over the data (185, 72), (170, 56), (168, 60), (179, 68), (182, 72), (188, 77) and show the clusters.

Solution:

- Given, number of clusters to be created (K) = 2 say c1 and c2.
- Initially choose first two objects as initial centroids.
- Let the centroid for first cluster c1 = (185, 72)
- And the centroid for second cluster c2 = (170, 56)
- Iteration 1:** calculate similarity by using *Euclidean distance* measure between every individual point and each centroid.
- Let d(c1, 3) be the distance between c1 and point 3 (168, 60)

Height	Weight
185	72
170	56
168	60
179	68
182	72
188	77

K-means Algorithm

$$d(c1, 3) = \sqrt{(185 - 168)^2 + (72 - 60)^2} = \sqrt{(17)^2 + (12)^2} = \sqrt{289 + 144} = \sqrt{433}$$

$$d(c2, 3) = \sqrt{(170 - 168)^2 + (56 - 60)^2} = \sqrt{(2)^2 + (-4)^2} = \sqrt{4 + 16} = \sqrt{20}$$

Here, $d(c2, 3) < d(c1, 3)$

So, data point 3 belongs to c2.

$$d(c1, 4) = \sqrt{(185 - 179)^2 + (72 - 68)^2} = \sqrt{(6)^2 + (4)^2} = \sqrt{36 + 16} = \sqrt{52}$$

$$d(c2, 4) = \sqrt{(170 - 179)^2 + (56 - 68)^2} = \sqrt{(-9)^2 + (-12)^2} = \sqrt{81 + 144} = \sqrt{225}$$

Here, $d(c1, 4) < d(c2, 4)$

So, data point 4 belongs to c1.

$$d(c1, 5) = \sqrt{(185 - 182)^2 + (72 - 72)^2} = \sqrt{(3)^2 + (0)^2} = \sqrt{9}$$

$$d(c2, 5) = \sqrt{(170 - 182)^2 + (56 - 72)^2} = \sqrt{(-12)^2 + (-16)^2} = \sqrt{144 + 256} = \sqrt{400}$$

Here, $d(c1, 5) < d(c2, 5)$

So, data point 5 belongs to c1.

$$d(c1, 6) = \sqrt{(185 - 188)^2 + (72 - 77)^2} = \sqrt{(-3)^2 + (-5)^2} = \sqrt{9 + 25} = \sqrt{34}$$

$$d(c2, 6) = \sqrt{(170 - 188)^2 + (56 - 77)^2} = \sqrt{(-18)^2 + (-21)^2} = \sqrt{324 + 441} = \sqrt{765}$$

Here, $d(c1, 6) < d(c2, 6)$

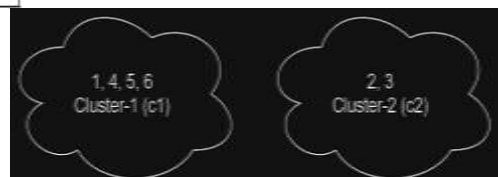
So, data point 6 belongs to c1.

K-means Algorithm

Tabulating the distances after iteration - 1:

Instance	X	Y	Distance(C1)	Distance(C2)	Cluster
1	185	72			c1
2	170	56			c2
3	168	60	$\sqrt{433}$	$\sqrt{20}$	c2
4	179	68	$\sqrt{52}$	$\sqrt{225}$	c1
5	182	72	$\sqrt{9}$	$\sqrt{400}$	c1
6	188	77	$\sqrt{34}$	$\sqrt{765}$	c1

The resulting cluster after first iteration is:



K-means Algorithm

- **Iteration 2:** Now calculate centroid for each cluster by finding the mean of all points in the cluster

$$\text{Centroid for first cluster } c1 = \left(\frac{185+179+182+188}{4}, \frac{72+68+72+77}{4} \right) = (183.5, 72.25)$$

$$\text{Centroid for second cluster } c2 = \left(\frac{170+168}{2}, \frac{56+60}{2} \right) = (169, 58)$$

- Calculate similarity by using *Euclidean distance* measure between every individual point and each centroid.
- Let $d(c1, 1)$ be the distance between $c1$ and point 1 (185, 72)

$$\text{Euclidean Distance between } A_1 \text{ and } B_2 = \sqrt{(X_2 - X_1)^2 + (Y_2 - Y_1)^2}$$

K-means Algorithm

$$d(c1, 1) = \sqrt{(183.5 - 185)^2 + (72.25 - 72)^2} = 1.5207$$

$$d(c2, 1) = \sqrt{(169 - 185)^2 + (58 - 72)^2} = 21.2603$$

Here, $d(c1, 1) < d(c2, 1)$

So, data point 1 belongs to $c1$.

$$d(c1, 2) = \sqrt{(183.5 - 170)^2 + (72.25 - 56)^2} = 21.1261$$

$$d(c2, 2) = \sqrt{(169 - 170)^2 + (58 - 56)^2} = 2.2361$$

Here, $d(c2, 2) < d(c1, 2)$

So, data point 2 belongs to $c2$.

$$d(c1, 3) = \sqrt{(183.5 - 168)^2 + (72.25 - 60)^2} = 19.7563$$

$$d(c2, 3) = \sqrt{(169 - 168)^2 + (58 - 60)^2} = 2.2361$$

Here, $d(c2, 3) < d(c1, 3)$

So, data point 3 belongs to $c2$.

$$d(c1, 4) = \sqrt{(183.5 - 179)^2 + (72.25 - 68)^2} = 6.1897$$

$$d(c2, 4) = \sqrt{(169 - 179)^2 + (58 - 68)^2} = 14.1421$$

Here, $d(c1, 4) < d(c2, 4)$

So, data point 4 belongs to $c1$.

$$d(c1, 5) = \sqrt{(183.5 - 182)^2 + (72.25 - 72)^2} = 1.5207$$

$$d(c2, 5) = \sqrt{(169 - 182)^2 + (58 - 72)^2} = 19.1050$$

Here, $d(c1, 5) < d(c2, 5)$

So, data point 5 belongs to $c1$.

$$d(c1, 6) = \sqrt{(183.5 - 188)^2 + (72.25 - 77)^2} = 6.5431$$

$$d(c2, 6) = \sqrt{(169 - 188)^2 + (58 - 77)^2} = 26.8701$$

Here, $d(c1, 6) < d(c2, 6)$

So, data point 6 belongs to $c1$.

K-means Algorithm

Tabulating the distances after iteration - 2:

Instance	X	Y	Distance(C1)	Distance(C2)	Cluster
1	185	72	1.5207	21.2603	c1
2	170	56	21.1261	2.2361	c2
3	168	60	19.7563	2.2361	c2
4	179	68	6.1897	14.1421	c1
5	182	72	1.5207	19.105	c1
6	188	77	6.5431	26.8701	c1

The resulting clusters after iteration - 2:

Same as that of after iteration-1.

Since, the clustering doesn't change after second iteration, terminate the iteration.



K-means Algorithm

Example - 2: Apply K-Means algorithm over the given data and show the clusters with $K = 2$.

No	X	Y
1	1	1
2	2	3
3	1	2
4	3	3
5	2	2
6	3	1

K-means Algorithm

Example - 3: Apply K-Means algorithm over the given data and show the clusters with $K = 3$.

Assume A1, A4, A7 as C1, C2, C3 initially.

POINT	X	Y
A1	2	10
A2	2	5
A3	8	4
A4	5	8
A5	7	5
A6	6	4
A7	1	2
A8	4	9

K-means Algorithm

Pseudocode

```
function kmeans(data_points, k):
    // Initialize centroids:
    // Randomly select k data points from 'data_points' as initial centroids
    centroids = select_random_centroids(data_points, k)

    converged = false
    while not converged:
        // Assign each data point to the nearest centroid:
        clusters = create_empty_clusters(k) // List of k empty lists
        for each point in data_points:
            min_distance = infinity
            closest_centroid_index = -1
            for i from 0 to k-1:
                distance = calculate_distance(point, centroids[i])
                if distance < min_distance:
                    min_distance = distance
                    closest_centroid_index = i
            add_point_to_cluster(point, clusters[closest_centroid_index])
```


K-means Algorithm

Pseudocode

```
// Recalculate centroids:
new_centroids = empty_list
for i from 0 to k-1:
    if clusters[i] is not empty:
        new_centroid = calculate_mean(clusters[i]) // Mean of all points in the cluster
        add_to_list(new_centroids, new_centroid)
    else:
        // Handle empty clusters (e.g., re-initialize centroid or leave as is)
        add_to_list(new_centroids, centroids[i]) // Keep old centroid if cluster is empty

// Check for convergence:
if new_centroids == centroids:
    converged = true
else:
    centroids = new_centroids

return clusters // Or centroids, depending on desired output
```

K-means Algorithm

Advantages

- **Simple and Easy to Implement:** K-Means is straightforward and easy to understand.
- **Efficient:** Computationally efficient when applied to large datasets.
- **Adaptability:** Works well when clusters are spherical and well-separated.

Limitations

- **Choice of K:** The number of clusters K needs to be specified before running the algorithm, which might not be intuitive.
- **Sensitive to Initialization:** The final clusters can depend on the initial selection of centroids, potentially leading to different results.
- **Convergence to Local Minima:** K-Means might converge to a local minimum, meaning that the solution might not be optimal.
- **Assumes Spherical Clusters:** K-Means assumes clusters are spherical in shape and similar in size, which may not always be the case.

K-means Algorithm

Applications

- **Customer Segmentation:** Grouping customers with similar behaviors.
- **Image Compression:** Reducing the number of colors in an image by clustering similar pixel values.
- **Document Clustering:** Organizing documents into related topics or categories.
- **Anomaly Detection:** Identifying unusual data points by looking for points far from cluster centroids.

INSTANCE-BASED LEARNING

Instance Based Learning

- It is a ML approach where the model makes predictions by storing and using instances of the training data directly, rather than constructing an explicit generalization (like a mathematical model).
- Instead of learning a function that maps inputs to outputs, the model learns based on specific examples from the training dataset and uses these instances to predict outcomes for new data points.
- Often referred to as "lazy learning" because there is minimal learning or processing done during training. The learning occurs at the time of prediction.
- Typically used for **supervised learning** tasks like classification and regression, where the training data contains labeled examples.
- Potentially computationally expensive.

Instance Based Learning

How Instance-Based Learning Works?

- When a new data point needs to be classified or predicted, the algorithm compares it to the stored instances from the training set.
- Based on the closest or most similar instances, the algorithm decides the prediction (classification or regression).

Different Algorithms of Instance-Based Learning:

- K-Nearest Neighbors (KNN) → predict based on majority vote/average of k closest neighbors
- Locally Weighted Learning (LWL) → fit a simple model using nearby points.
- **Case-Based Reasoning** → solve new problems by adapting solutions from similar past cases.

Instance Based Learning

- **KNN algorithm**

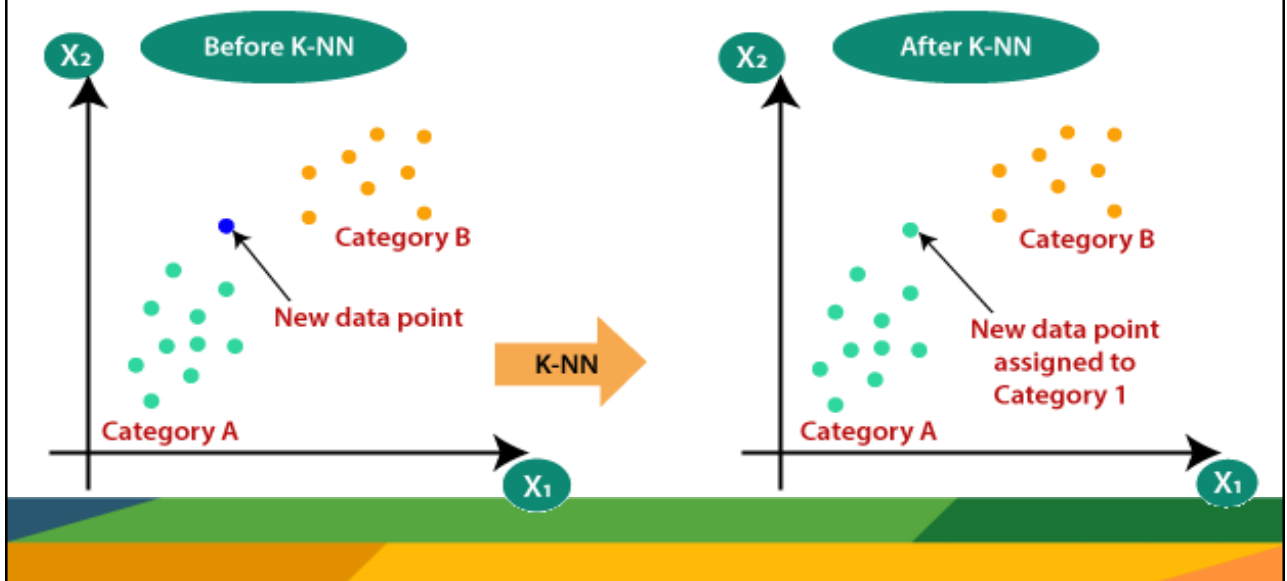
KNN Algorithm

- KNN → K Nearest Neighbors → the most widely known instance-based learning algorithm.
- A non-parametric (does not make any assumption on underlying data), supervised learning classifier, which uses proximity to make classifications or predictions about the grouping of an individual data point.
- It is one of the popular and simplest classification and regression classifiers used in machine learning today.
- It classifies a new data point by finding the "k" closest instances in the training set (based on some distance metric) and assigns the class most common among those neighbors.
- For instance, if we want to classify a data point, we would look at its k-nearest neighbors and use their labels to decide which class the data point should belong to.

KNN Algorithm

- K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm.
- K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.
- It is also called a **lazy learner algorithm** because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.
- KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.

KNN Algorithm



KNN Algorithm

How does K-NN work?

To assign class label for a new data point P

Step-1: Select the number K of the neighbors

Step-2: Calculate the Euclidean distance of K number of neighbors from point P

Step-3: Take the K nearest neighbors as per the calculated Euclidean distance.

Step-4: Among these k neighbors, count the number of the data points in each category.

Step-5: Assign the new data points to that category for which the number of the neighbor is maximum

KNN Algorithm

Algorithm

Algorithm K-Nearest Neighbors (KNN) for Classification

Input:

- Training_Data: A set of labeled data points (features, class_label)
- Test_Point: An unlabeled data point (features) for which to predict the class
- K: The number of nearest neighbors to consider

Output:

- Predicted_Class: The predicted class label for Test_Point

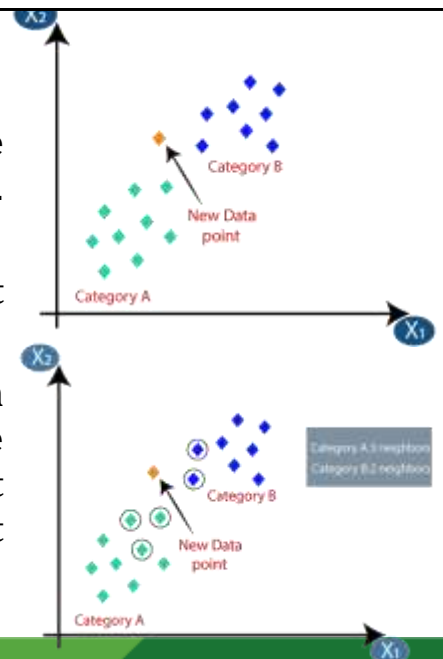
KNN Algorithm

Steps:

1. Initialize an empty list called 'distances'.
2. For each data_point in Training_Data:
 - a. Calculate the distance between Test_Point and data_point using a chosen distance metric (e.g., Euclidean distance, Manhattan distance).
 - b. Store the calculated distance along with the class_label of data_point in the 'distances' list.
3. Sort the 'distances' list in ascending order based on the calculated distances.
4. Select the first K entries from the sorted 'distances' list. These represent the K-nearest neighbors.
5. Extract the class labels of these K-nearest neighbors.
6. Determine the most frequent class label among the extracted class labels (e.g., using majority vote).
7. Assign this most frequent class label as the Predicted Class for Test Point.

KNN Algorithm

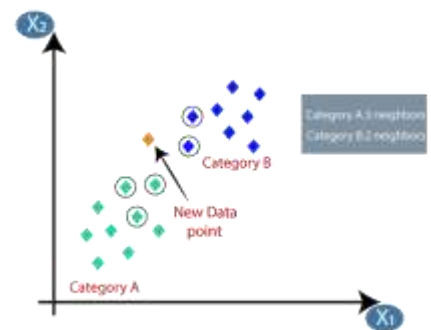
- Suppose we have a new data point and we need to put it in the required category. Consider the given image:
- Firstly, choose the number of neighbors k, let $k=5$.
- Next, calculate the **Euclidean distance** between the data points to get the nearest neighbors, as three nearest neighbors in category A and two nearest neighbors in category B.



$$\text{Euclidean Distance between } A_1 \text{ and } B_2 = \sqrt{(X_2 - X_1)^2 + (Y_2 - Y_1)^2}$$

KNN Algorithm

- Since the 3 nearest neighbors are from category A, this new data point must belong to category A.
- There is no particular way to determine the best value for "K", so need to try some values to find the best out of them. The most preferred value for K is 5.
- A very low value for K such as $K=1$ or $K=2$, can be noisy and lead to the effects of outliers in the model.
- Large values for K are good, but it may find some difficulties.



KNN Algorithm

Advantages

- Simple to implement.
- Robust to the noisy training data
- Can be more effective if the training data is large.

Limitations

- Always needs to determine the value of K which may be complex some time.
- The computation cost is high because of calculating the distance between the data points for all the training samples.
- A lot of memory is required for processing large data sets.

KNN Algorithm

Example problem:

- Consider two columns — Brightness and Saturation. Each row in the table has a class of either Red or Blue. Find the class for the last entry, with Brightness=20 and Saturation=35. Assume $K = 5$.

Solution:

- To know the class for the new entry, calculate the distance from the new entry to other entries in the data set using the Euclidean distance formula.
- For the first row, $d_1 = \sqrt{(20 - 40)^2 + (35 - 20)^2} = \sqrt{400 + 225} = \sqrt{625} = 25$
- Similarly find the other distances and tabulate

Brightness	Saturation	Class
40	20	Red
50	50	Blue
60	90	Blue
10	25	Red
70	70	Blue
60	10	Red
25	80	Blue
20	35	??

KNN Algorithm

Brightness	Saturation	Class	Distance
40	20	Red	25
50	50	Blue	33.54
60	90	Blue	68.01
10	25	Red	14.14
70	70	Blue	61.03
60	10	Red	47.17
25	80	Blue	45
20	35	??	

Rearrange the distances in ascending order:

Brightness	Saturation	Class	Distance
10	25	Red	14.14
40	20	Red	25
50	50	Blue	33.54
25	80	Blue	45
60	10	Red	47.17
70	70	Blue	61.03
60	90	Blue	68.01
20	35	??	

KNN Algorithm

Consider first 5 rows (as $K = 5$):

Since the majority class within the 5 nearest neighbors to the new entry is Red, the new entry will be classified as Red.

Brightness	Saturation	Class	Distance	Brightness	Saturation	Class	Distance
10	25	Red	14.14	10	25	Red	14.14
40	20	Red	25	40	20	Red	25
50	50	Blue	33.54	50	50	Blue	33.54
25	80	Blue	45	25	80	Blue	45
60	10	Red	47.17	60	10	Red	47.17
20	35	??		20	35	Red	

KNN Algorithm

Example 2: Given the dataset below, use **K-Nearest Neighbors** with $K=3$ to predict the output at $x=4.2$ using unweighted and weighted KNN.

X	1	2	3	5	6	7	4.2
y	2.0	3.0	2.5	4.0	4.5	5.0	?

Solution:

Compute the distances to x:

$$d(1,4.2) = 3.2 \quad d(2,4.2) = 2.2 \quad d(3,4.2) = 1.2$$

$$d(5,4.2) = 0.8 \quad d(6,4.2) = 1.8 \quad d(7,4.2) = 2.8$$

Nearest 3 neighbors:

1. $x=5, y=4.0$ (distance 0.8)
2. $x=3, y=2.5$ (distance 1.2)
3. $x=6, y=4.5$ (distance 1.8)

KNN Algorithm

1. Unweighted KNN prediction $\hat{y} = \frac{4.0+2.5+4.5}{3} = \frac{11}{3} \approx 3.667$

2. Distance-weighted KNN (weights $w=1/d^2$)

Nearest 3 neighbors: $x=5, y=4.0$ (distance 0.8)
 $x=3, y=2.5$ (distance 1.2) $x=6, y=4.5$ (distance 1.8)

Weights:

$$w_{0.8} = 1/0.8^2 = 1.5625$$

$$w_{1.2} = 1/1.2^2 \approx 0.6944$$

$$w_{1.8} = 1/1.8^2 \approx 0.3086$$

Weighted KNN prediction

$$\hat{y} = \frac{1.5625 \times 4.0 + 0.6944 \times 2.5 + 0.3086 \times 4.5}{1.5625 + 0.6944 + 0.3086} \approx \frac{9.375}{2.5656} \approx 3.654$$

GAUSSIAN MIXTURE MODELS

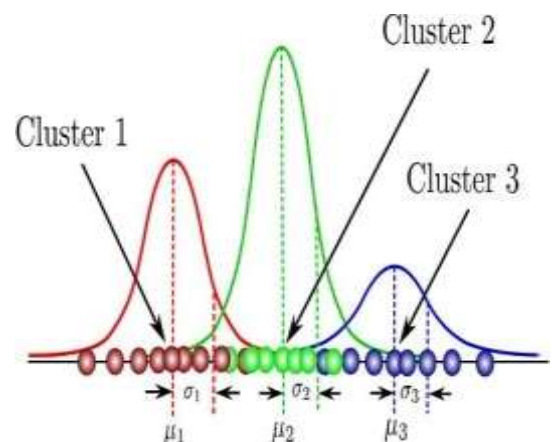
Gaussian Mixture Models

- A Gaussian mixture model is a **soft clustering** machine learning method used to determine the probability each data point belongs to a given cluster.
- Gaussian distribution (also known as normal distribution) is a bell-shaped curve, and it is assumed that during any measurement, values will follow a **normal distribution** with **an equal number of measurements above and below the mean value**.
- Used to describe collections of independent values (data sets) obtained from measurements on a defined population of samples.
- If data can be modeled with a Gaussian distribution, then levels of confidence can be associated with the results based on the average value and distance from it using standard deviation values.

Gaussian Mixture Models

- A Gaussian mixture is a function that is composed of several Gaussians, each identified by $k \in \{1, \dots, K\}$, where K is the number of clusters of our data set. Each Gaussian k in the mixture is comprised of the following parameters:
 - A mean μ that defines its center.
 - A covariance Σ that defines its width.
 - A mixing probability π that defines how big or small the Gaussian function will be.

A GMM with 3 Gaussian functions, $K=3$



Gaussian Mixture Models

- Each Gaussian explains the data contained in each of the three clusters available.
- The mixing coefficients are themselves probabilities and must meet this condition:

$$\sum_{k=1}^K \pi_k = 1$$

- For eg., if $K=3$, the probability that a data belongs to Gaussian1, Gaussian2 and Gaussian3 will be π_1 , π_2 and π_3 respectively, and $\pi_1 + \pi_2 + \pi_3 = 1$

Gaussian Mixture Models

- In one dimension the probability density function of a Gaussian Distribution is given by

$$G(X|\mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Where $x \rightarrow$ the scalar data point,

$\mu \rightarrow$ the mean of the Gaussian component,

$\sigma^2 \rightarrow$ the variance of the Gaussian component.

- For a mixture of K Gaussian components, the probability of x under the mixture model is:

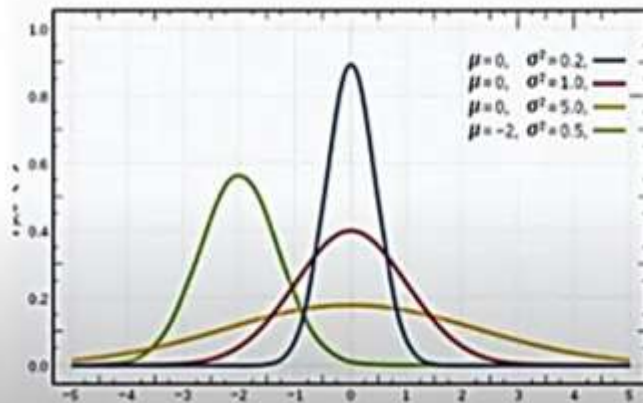
$$p(x|\theta) = \sum_{k=1}^K \pi_k \cdot p(x|\mu_k, \sigma_k^2)$$

Where:

- π_k is the mixing coefficient for the k -th component, with $\sum_{k=1}^K \pi_k = 1$ and $\pi_k \geq 0$,
- $p(x|\mu_k, \sigma_k^2)$ is the probability of x under the k -th Gaussian component,
- $\theta = \{\pi_k, \mu_k, \sigma_k^2\}_{k=1}^K$ is the set of all parameters.

Gaussian Mixture Models

GMM For 1 variable



$$f(x | \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Gaussian Mixture Models

- For Multivariate (let us say d-variate) Gaussian Distribution, the probability density function is given by $G(X|\mu, \Sigma) = \frac{1}{\sqrt{(2\pi)^d |\Sigma|}} \exp\left(-\frac{1}{2}(X - \mu)^T \Sigma^{-1} (X - \mu)\right)$
- Suppose there are K clusters.
- So μ and Σ are also estimated for each k.
- Only one distribution $\rightarrow$ clusters would have been estimated by the maximum-likelihood method.
- But since K such clusters and the probability density is defined as a linear function of densities of all these K distributions, i.e.

$$p(X) = \sum_{k=1}^K \pi_k G(X|\mu_k, \Sigma_k)$$

- Where π_k is the mixing coefficient for k^{th} distribution, μ_k is the mean vector, Σ_k is the covariance matrix

Gaussian Mixture Models

- Gaussian Mixture Models is a “soft” clustering algorithm, where each point probabilistically “belongs” to all clusters.
- This is different than k-means where each point belongs to one cluster.
- The GMM is a probabilistic model that assumes all the data points are generated from a mix of limited set of Gaussian distributions with unknown parameters.
- The mean and covariance matrix characterizes each individual Gaussian distribution.
- GMM do not rigidly classify each and every instance into one class or the other.
- The algorithm attempts to produce K-Gaussian distributions that would take into account the entire training space.
- Every point can be associated with one or more distributions.
- Consequently, the deterministic factor would be the probability that each point belongs to a certain Gaussian distribution.

Gaussian Mixture Models

- Gaussian Mixture Models is a “soft” clustering algorithm → This provides more flexibility and may accommodate scenarios when data points do not naturally fall into one cluster.
- The GMM is trained using the Expectation Maximization (EM) algorithm → an iterative approach for determining the most likely estimations of the mixture Gaussian distribution parameters.
- The EM method first makes rough guesses at the parameters, then repeatedly improves those guesses until convergence is reached.

Gaussian Mixture Models

How GMM algorithm works:

1. **Initialize phase:** Gaussian distributions' parameters should be initialized (means, covariances, and mixing coefficients).
2. **Expectation phase:** Determine the likelihood that each data point was created using each of the Gaussian distributions.
3. **Maximization phase:** Apply the probabilities found in the expectation step to re-estimate the Gaussian distribution parameters.
4. **Final phase:** To achieve convergence of the parameters, repeat steps 2 and 3.

GAUSSIAN MIXTURE MODELS

- **Expectation Maximization**

Expectation Maximization Algorithm

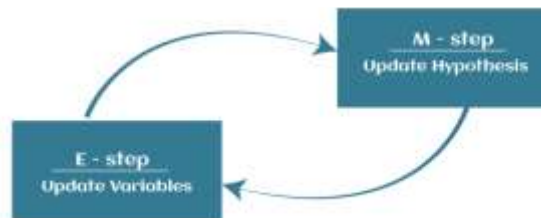
- GMM used for clustering and GMM uses EM algorithm.
- To cluster, try to fit a probability density function for the training data.
- Maximum Likelihood Estimation (MLE) technique can be used → a statistical method used to estimate the parameters of a model → identifies the parameters that make the observed data most probable under the assumed statistical model.
- But MLE requires all variables that have impact on clustering
- In real time, observed variables and unobservable (or) latent variables exist
- For eg. in predicting the student's performance in an exam, based on no. of hours studied, no. of revision tests, etc., the level of understanding may be a latent variable.
- EM can be used to estimate the values of latent variables → EM is an approach for performing MLE in the presence of latent variables → also referred to as the latent variable model.

Expectation Maximization Algorithm

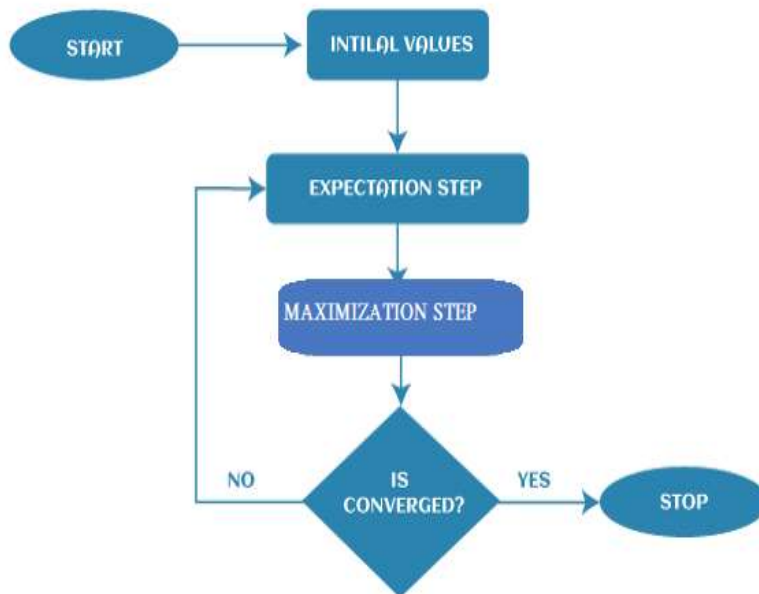
- The EM algorithm is considered a latent variable model to find the local maximum likelihood parameters of a statistical model, proposed by Arthur Dempster, Nan Laird, and Donald Rubin in 1977.
- The Expectation-Maximization (EM) algorithm is defined as the combination of various unsupervised machine learning algorithms, which is used to determine the local maximum likelihood estimates (MLE) or maximum a posteriori estimates (MAP) for unobservable variables in statistical models.
- Further, it is a technique to find maximum likelihood estimation when the latent variables are present. It is also referred to as the latent variable model.

Expectation Maximization Algorithm

- The EM algorithm is the combination of various unsupervised ML algorithms, such as the k-means clustering algorithm.
- Being an iterative approach, it consists of two steps.
- Expectation step (E step) – Estimate latent variables based on known / observed variables
- Maximization step (M step) - Update the probability distribution parameters using the estimated data.
- Repeat E-step and M-step until the convergence of the values occurs.



Expectation Maximization Algorithm



Expectation Maximization Algorithm

Steps involved in EM algorithm

1. Given a set of incomplete data, consider a set of starting parameters and initialize them.
2. Expectation step (E step) – using the observed available data of the dataset, estimate the values of the missing data.
3. Maximization step (M step) – the complete data generated after the E-step is used to update the parameters.
4. Repeat steps 2 and 3 until convergence.

Expectation Maximization Algorithm

E-step:

For each point x_i , calculate the probability that it belongs to cluster/distribution $c_1, c_2, \dots, c_k$. This is done using the below formula:

$$r_{ic} = \frac{\text{Probability } x_i \text{ belongs to } c}{\text{Sum of probability } x_i \text{ belongs to } c_1, c_2, \dots, c_k} = \frac{\pi_c \mathcal{N}(x_i; \mu_c, \Sigma_c)}{\sum_{c'} \pi_{c'} \mathcal{N}(x_i; \mu_{c'}, \Sigma_{c'})}$$

This value will be high when the point is assigned to the right cluster and lower otherwise.

Expectation Maximization Algorithm

M-step:

Post the E-step, we go back and update the Π , μ and Σ values. These are updated in the following manner:

1. The new density is defined by the ratio of the number of points in the cluster and the total number of points:

$$\Pi = \frac{\text{Number of points assigned to cluster}}{\text{Total number of points}}$$

2. The mean and the covariance matrix are updated based on the values assigned to the distribution, in proportion with the probability values for the data point. Hence, a data point that has a higher probability of being a part of that distribution will contribute a larger portion:

$$\mu = \frac{1}{\text{Number of points assigned to cluster}} \sum_i r_{ik} x_i$$

$$\Sigma_k = \frac{1}{\text{Number of points assigned to cluster}} \sum_i r_{ik} (x_i - \mu_k)^T (x_i - \mu_k)$$

Based on the updated values generated from this step, calculate the new probabilities for each data point and update the values iteratively.

Expectation Maximization Algorithm

Usage of EM algorithm

- Used to fill the missing data in a sample.
- Used as the basis of unsupervised learning of clusters.
- Used to discover the values of latent variables.

Advantages of EM algorithm

- It is very easy to implement the first two basic steps of the EM algorithm in various machine learning problems, which are E-step and M- step.
- It is mostly guaranteed that likelihood will enhance after each iteration.
- It often generates a solution for the M-step in the closed form.

Disadvantages of EM algorithm

- Has slow convergence.
- It can make convergence for the local optima only.
- It requires both forward and backward probabilities into consideration (whereas numerical optimization requires forward probabilities).